

PCM

Problème du voyageur de commerce

2025-01-05

Loïc Frossard & Baptiste Dupertuis

HES-SO Master

2024-2025

Rapport

Table des matières

1 Introduction	3
1.1 Objectifs	3
2 Structure du programme	3
2.1 Schéma block	4
3 Analyse des résultats	5
3.1 Sans condition de fin sure (version 1)	5
3.1.1 Speedup et efficience	5
3.2 Avec condition de fin sure (version 2)	6
3.2.1 Speedup et efficience	6
3.3 Analyse des performances des deux versions	7
3.3.1 Temps d'exécution	7
3.3.2 Écart type sur le temps d'exécution des threads	7
3.3.3 Analyse de l'effet de MAX_DEPTH	8
3.3.4 Analyse de la concurrence sur la queue	8
4 Conclusion	9
4.1 Améliorations possibles	9

1 Introduction

Ce rapport présente le projet final du cours de Programmation Concurrente et Multicœur (PCM). Le projet réalisé est capable de résoudre le Problème du voyageur de commerce (TSP, Traveling Salesman Problem) pour 19 villes en environ 8 minutes 37 secondes sur 196 threads.

1.1 Objectifs

L'objectif de ce projet est de développer un programme qui profite d'une architecture multicœur, en utilisant les techniques définies durant le cours de PCM.

Le travail consiste à résoudre le Problème du voyageur de commerce (TSP, Traveling Salesman Problem) avec une méthode branch-and-bound. Le programme doit visiter N villes une seule fois et doit revenir à la ville de départ, en empruntant le chemin le plus court. Étant donné que le chemin est circulaire, le choix de la ville de départ n'a pas d'importance. Ceci est un problème difficile d'optimisation combinatoire, car s'il y a N villes alors, au départ d'une ville donnée, il y a $(N-1)!$ circuits différents qui passent par les $N-1$ autres villes et reviennent à la ville de départ. Il est supposé qu'il existe un chemin indépendant entre toute paire de villes.

2 Structure du programme

Le programme se base sur l'utilisation d'une méthode branch-and-bound pour résoudre le problème du voyageur de commerce. La méthode branch-and-bound est une technique de résolution de problèmes d'optimisation combinatoire. Elle consiste à diviser le problème en sous-problèmes plus petits, à évaluer ces sous-problèmes et à éliminer les branches qui ne peuvent pas contenir la solution optimale.

Pour ce faire, le programme utilise une approche parallèle, en divisant le problème en plusieurs sous-problèmes qui sont résolus en parallèle. Lorsqu'un problème est résolu, le programme vérifie si la solution trouvée est meilleure que la meilleure solution actuelle. Si c'est le cas, la meilleure solution est mise à jour. Les autres sous-problèmes sont ensuite évalués en fonction de la meilleure solution actuelle, afin d'éliminer les branches qui ne peuvent pas contenir la solution optimale. Ce processus est répété jusqu'à ce que tous les sous-problèmes soient résolus.

Le programme est écrit en C++ et utilise la bibliothèque `atomic` pour gérer les accès concurrents aux données partagées. Il utilise également la bibliothèque `thread` pour créer et gérer les threads. Le programme prend en entrée un fichier contenant les coordonnées des villes à visiter et affiche en sortie le chemin optimal ainsi que la distance totale parcourue.

La communication des problèmes entre les threads se fait à l'aide d'une queue. Cette queue n'utilise pas de lock pour garantir l'accès exclusif aux données partagées, mais utilise des opérations atomiques (notamment des CAS) pour garantir la cohérence des données. L'implémentation de cette queue se base sur celle fournie dans le cours de PCM.

L'indication de la meilleure solution est faite à l'aide d'une variable partagée, cette variable est mise à jour de manière atomique pour garantir sa cohérence.

2.1 Schéma block

Le schéma suivant illustre le fonctionnement du programme. Ici la condition de fin pour un thread est la suivante : si la queue est vide, le thread s'arrête. Cette condition de fin n'est pas sûre, car il est possible que d'autres threads ajoutent des éléments à la queue après que le thread ait vérifié qu'elle était vide. Cela peut entraîner une situation où un thread s'arrête alors qu'il reste des éléments à traiter. Cependant, dans la pratique, cette situation ne s'est pas produite lors de nos tests. Nous avons aussi analysé un programme avec une condition de fin sûre et analysé l'impact sur les performances. La condition de fin sûre se base sur le comptage des chemins traités.

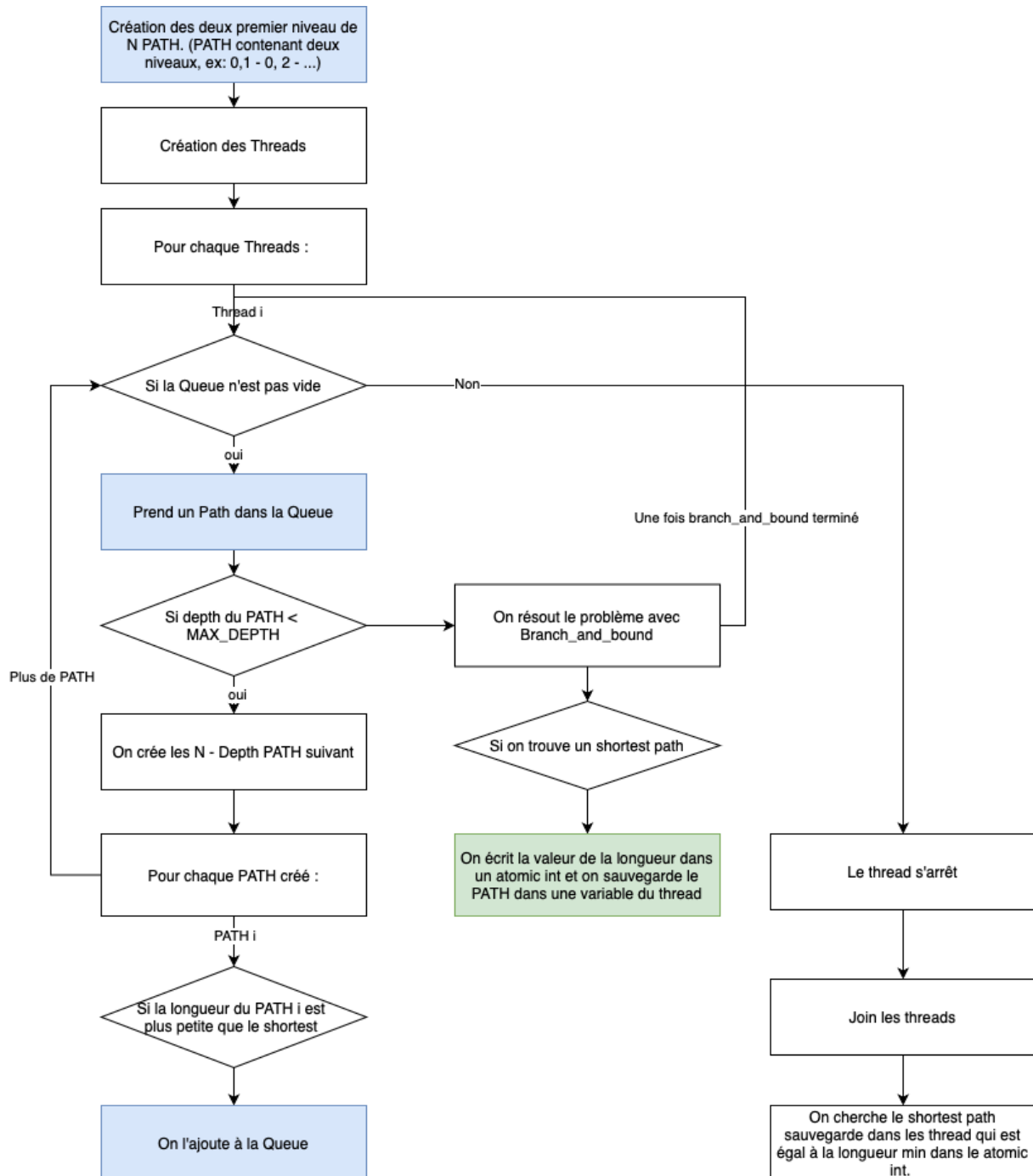


Fig. 1 — Schéma de fonctionnement du programme

Les cases bleues représente les actions qui interagissent avec la queue, les cases vertes représentent les actions qui interagissent avec la variable de la meilleure solution en écriture.

Voici les points clés du schéma :

- Le programme commence par initialiser la queue avec les deux ou trois premiers niveaux de l'arbre de recherche. Cela permet de créer assez de travail pour tous les threads et d'éviter un démarrage lent.
- Chaque thread récupère un sous-problème de la queue, si le sous-problème est suffisamment petit (il contient MAX_DEPTH villes), le thread le résout et met à jour la meilleure solution si il en trouve une. Si le problème est trop grand, le thread crée le prochain niveau de l'arbre de recherche et ajoute les sous-problèmes à la queue.
- Lorsqu'un thread a terminé de traiter un sous-problème, il en récupère un autre de la queue. Dans la version 1 du code, si la queue est vide, le thread s'arrête. Dans la version 2, le thread s'arrête uniquement si le nombre de sous-problèmes traités est égal au nombre total de sous-problèmes à traiter.

3 Analyse des résultats

Ce chapitre présente les résultats obtenus lors de l'exécution des deux versions du programme (avec et sans condition de fin sûre). Les résultats sont basés sur des tests effectués sur un serveur de calcul avec 256 threads. Les tests ont été effectués pour un nombre de villes allant de 10 à 18, et des threads allant de 32 à 256 par pas de 32. Les résultats sont basés sur la moyenne de 10 exécutions de chaque test. Les temps de références pour la meilleure version séquentielle sont donné

dans le tableau ci-dessous. Le programme de référence est celui de monsieur M. Passin avec la dernière version de la classe Path. Ces temps sont la moyenne de 2 mesures.

Nb villes	Temps [ms]
13	3992
14	8622
15	46393
16	228210
17	1137842
18	6541798

3.1 Sans condition de fin sure (version 1)

Ce chapitre présente les résultats obtenus lors de l'exécution de la première version du programme, sans condition de fin sûre.

3.1.1 Speedup et efficience

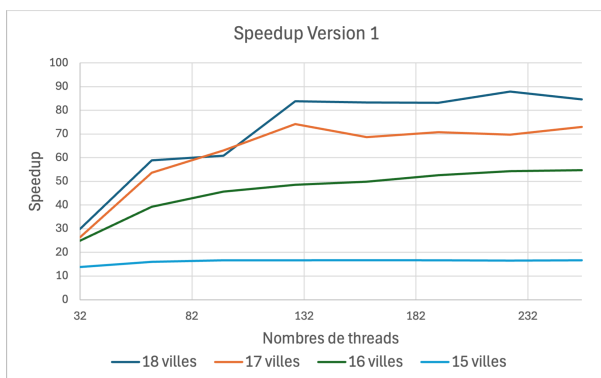


Fig. 2 — Speedup en fonction du nombre de villes - Version 1

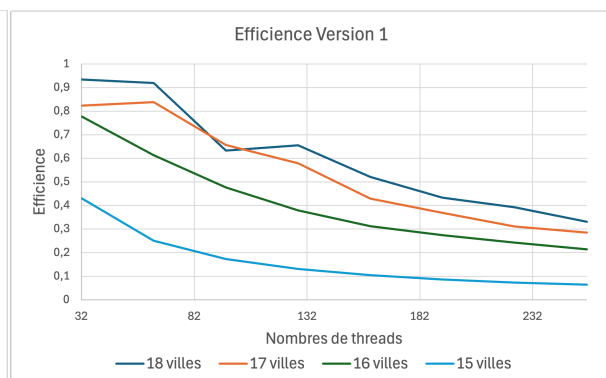


Fig. 3 — Efficience en fonction du nombre de villes - Version 1

On peut voir que le speedup augmente avec le nombre de villes, mais que l'efficacité diminue. De même le speedup augmente avec le nombre de threads, jusqu'à atteindre un plateau. Cela est dû au fait que le problème devient plus complexe avec un plus grand nombre de villes, ce qui permet de mieux exploiter les ressources des threads. Cependant, l'efficacité diminue car le temps de communication entre les threads devient plus important par rapport au temps de calcul, ce qui limite les gains de performance.

Le speedup maximale est atteint pour 224 threads et 18 villes, avec un speedup de 87. L'efficacité maximale est atteinte pour 32 threads et 18 villes, avec une efficacité de 0.90.

3.2 Avec condition de fin sûre (version 2)

Ce chapitre présente les résultats obtenus lors de l'exécution de la deuxième version du programme, avec condition de fin sûre.

3.2.1 Speedup et efficacité

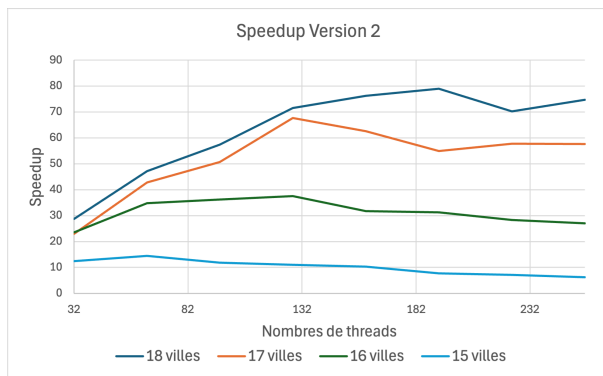


Fig. 4 – Speedup en fonction du nombre de villes - Version 2

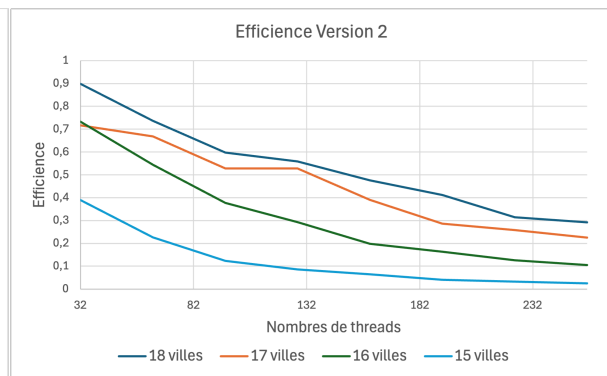


Fig. 5 – Efficacité en fonction du nombre de villes - Version 2

Les observations sont similaires à la version 1. Cependant le speedup maximum est plus faible (il est au maximum de 78 pour 192 threads et 18 villes). Cela est dû au fait que la condition de fin sûre augmente légèrement la charge de travail des threads car ils doivent comptabiliser les sous-problèmes traités. Cela peut entraîner une légère baisse de performance, mais cela permet de garantir que tous les threads s'arrêtent uniquement lorsque tous les sous-problèmes ont été traités.

Le calcul des paths traités utilise une table pré-remplie des 20 premières factorielles, ce qui permet de réduire le temps de calcul pour déterminer le nombre de problèmes traités lors de l'élimination d'un sous-problème.

3.3 Analyse des performances des deux versions

Ce chapitre analyse les performances des deux versions et aborde certains points comme la concurrence sur la queue et la taille des sous-problème finaux.

3.3.1 Temps d'exécution

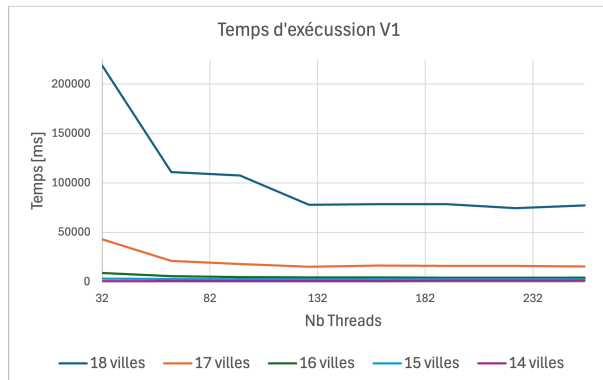


Fig. 6 — Temps d'exécution en fonction du nombre de villes - Version 1

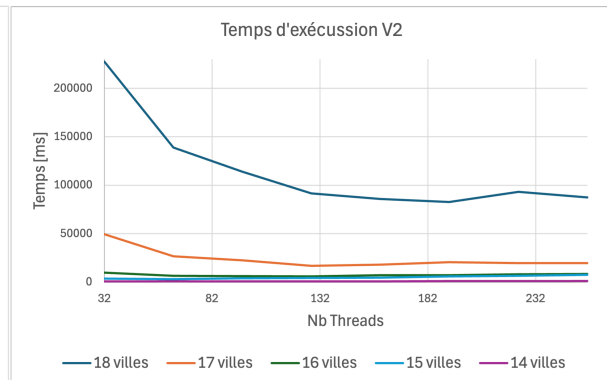


Fig. 7 — Temps d'exécution en fonction du nombre de villes - Version 2

Dans les figures ci-dessus, on constate que le temps d'exécution entre les deux versions est relativement semblable. Or si on compare les speedup, on remarque la version 2 est presque 10 fois plus lente dans certains cas. Cela s'explique par le travail supplémentaire qu'il faut réaliser pour déterminer la fin sure

de l'algorithme. Le tableau suivant montre les temps réalisés pour 19 villes.

Version	Nb threads	Temps [ms]
V1	196	517276 (8m37s)
V2	x	x

3.3.2 Écart type sur le temps d'exécution des threads

L'image suivante montre l'écart type sur le temps d'exécution des threads pour les deux versions du programme. On peut voir que la version 1 a un écart type plus élevé que la version 2. Cela est dû au fait que dans la version 1, les threads s'arrêtent dès que la queue est vide, ce qui peut entraîner des différences de temps d'exécution entre les threads. Dans la version 2, les threads s'arrêtent uniquement lorsque le nombre de sous-problèmes traités est égal au nombre total de sous-problèmes à traiter, ce qui permet de réduire l'écart type sur le temps d'exécution des threads. Cependant, **l'écart type reste relativement faible pour les deux versions, ce qui tend à montrer que les threads sont bien équilibrés en termes de charge de travail, même dans la version 1.** Un grand pic est visible dans la version 1 aux alentours de 15 villes et qui dimi-

nue pour les nombres de villes supérieurs. Ce pic est probablement dû au fait que pour 15 villes le temps d'exécution total est relativement court et donc l'écart type est plus visible car le problème est de taille modérée. Pour les villes inférieures, le temps d'exécution est très (trop) court, donc l'écart type est faible et peu visible.

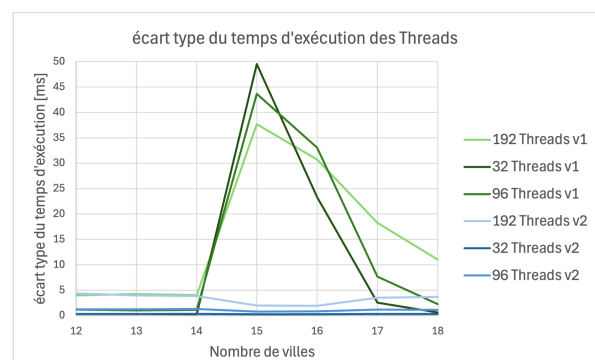


Fig. 8 — Écart type sur le temps d'exécution des threads

3.3.3 Analyse de l'effet de MAX_DEPTH

L'analyse suivante se base sur la version 1 du programme. MAX_DEPTH est déterminée par le nombre de villes et le nombre de niveaux que l'on veut laisser résoudre à un thread. Les deux graphes ci-dessus montrent l'impact de MAX_DEPTH sur le temps d'exécution et l'écart-type du temps d'exécution des threads. Lorsqu'un problème atteint MAX_DEPTH, il est résolu par un thread (c'est à dire lorsqu'il reste $N - \text{MAX_DEPTH}$ niveau à explorer, où N est le nombre de villes).

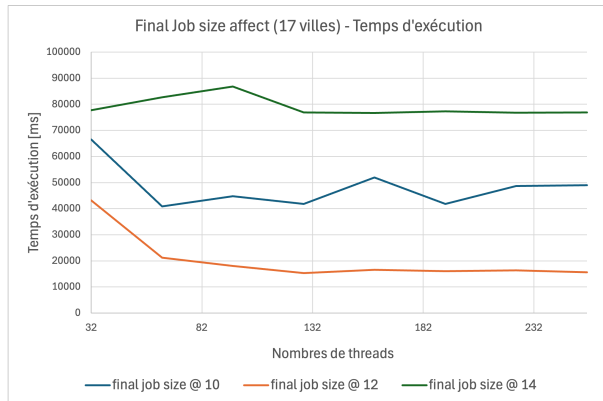


Fig. 9 — Taille des sous-problèmes finaux - temps d'exécution

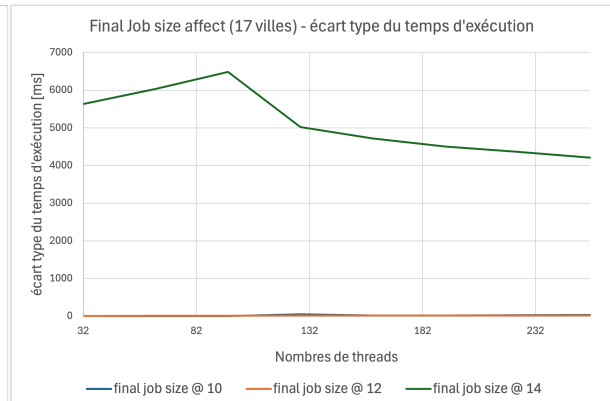


Fig. 10 — Taille des sous-problèmes finaux - écart-type

On constate que si le problème final est trop grand (MAX_DEPTH trop petit), le temps d'exécution augmente et l'écart-type augmente aussi considérablement. Cela est dû au fait que les threads passent plus de temps à résoudre un sous-problème, ce qui augmente la variabilité du temps d'exécution des threads (si il n'y a plus de travail certains threads s'arrêtent, alors que ceux qui ont encore du travail vont mettre du temps à le finir). En revanche, si le problème final est trop petit (MAX_DEPTH trop grand), le temps d'exécution augmente aussi mais l'écart-type diminue. Cela est dû au fait que les threads passent moins de temps à résoudre un sous-problème, mais plus de temps à chercher du travail dans la queue. Cela diminue la variabilité du temps d'exécution des threads, car lorsqu'il n'y a plus de travail, les threads qui en ont encore le finissent rapidement.

Pour conclure le choix de la taille du travail final est important pour les performances du programme. Il est important de trouver un équilibre entre le temps de calcul et le temps de recherche de travail pour les threads. Dans les tests réalisés, une taille de 12 villes pour le travail final semble être un bon compromis.

3.3.4 Analyse de la concurrence sur la queue

Les deux extraits de logs ci-dessous montrent la concurrence sur la queue pour deux graphes de taille 17 et 18. On peut voir que la concurrence est relativement faible pour les deux graphes, avec un maximum de 35 et 2583 accès concurrents pour un thread. Sachant que dans le premier cas il y a $(17-1)!$ problèmes et $(18-1)!$ problèmes pour le deuxième. Cependant, il est important de noter qu'il n'est pas possible de comparer directement le nombre de problèmes avec le nombre d'accès concurrents, car tous les sous-problèmes ne sont pas mis dans la queue.

Ici un accès concurrent pour un thread montre que le thread a accédé à la queue en même temps qu'un autre thread et qu'il a dû recommencer son accès car la queue, car le CAS a

échoué. Cela montre que la queue est bien utilisée par les threads et que la concurrence est bien gérée.

Il est important de noter que la valeur de MAX_DEPTH a un impact significatif sur le nombre d'accès concurrents. En effet, plus MAX_DEPTH est grand, plus il y a de sous-problèmes mis dans la queue et donc plus il y a d'accès fréquent à celle-ci.

```
Graph size: 17
Total Time: 21249 milliseconds
Thread:0      concurrency:35      duration (ms):21247
Thread:1      concurrency:16      duration (ms):21248
Thread:2      concurrency:22      duration (ms):21247
Thread:3      concurrency:24      duration (ms):21247
Thread:4      concurrency:23      duration (ms):21247
Thread:5      concurrency:21      duration (ms):21247
Thread:6      concurrency:28      duration (ms):21247
Thread:7      concurrency:31      duration (ms):21248
Thread:8      concurrency:25      duration (ms):21246
Thread:9      concurrency:35      duration (ms):21247
Standard deviation : 0.538516
Final problem size : 12
End condition : empty queue
shortest [3249: 0, 1, 3, 2, 4, 5, 6, 7, 8, 11, 10, 16, 15, 12, 14, 13, 9, 0]
```

```
Graph size: 18
Total Time: 126947 milliseconds
Thread:0      concurrency:2583      duration (ms):126947
Thread:1      concurrency:2220      duration (ms):126947
Thread:2      concurrency:2210      duration (ms):126946
Thread:3      concurrency:2292      duration (ms):126946
Thread:4      concurrency:2002      duration (ms):126946
Thread:5      concurrency:1974      duration (ms):126946
Thread:6      concurrency:2314      duration (ms):126946
Thread:7      concurrency:2443      duration (ms):126946
Thread:8      concurrency:1822      duration (ms):126944
Thread:9      concurrency:1985      duration (ms):126943
Standard deviation : 1.18743
Final problem size : 12
End condition : empty queue
shortest [3270: 0, 1, 3, 2, 4, 5, 6, 7, 8, 11, 10, 16, 17, 15, 12, 14, 13, 9, 0]
```

4 Conclusion

Pour conclure, le programme réalisé permet de résoudre le problème du voyageur de commerce avec une méthode branch-and-bound en 8 minutes 37 secondes avec 19 villes et 196 threads. Le meilleur speedup calculé est de 87 pour 18 villes et 224 threads. L'utilisation de méthode d'accès concurrent sans lock au données partagées était le point central de ce projet. Dans le travail réalisé deux éléments principaux ont utilisé ces méthodes, une queue pour communiquer entre les threads et une variable partagée pour communiquer la meilleure solution.

Deux approches sur la manière d'arrêter les threads ont été analysées. Une première non sure, qui arrête un threads lorsque le queue est vide et une deuxième sure qui arrête un threads lorsque tout le travail a été réalisé. La première solution fonctionne et d'expérience les threads ne s'arrête pas de manière prématurée, cependant il n'as pas été prouvé que c'est le cas à chaque fois. L'ajout de la condition de fin sure ralentit le programme de près de 10 fois. En effet, le meilleur speedup trouvé avec cette solution est de 78 pour 192 threads et 18 villes.

Finalement, le choix de la taille des problèmes finaux calculé par un thread a été analysé. Le choix de cette valeur a un grand impact sur le temps d'exécution total et sur le nombre d'accès concurrent sur la queue. La meilleure valeur semble être 12, c'est à dire qu'un thrad résout l'entièreté du sous-problème sur il ne manque que 12 villes à celui-ci.

4.1 Améliorations possibles

Une amélioration possible serait l'utilisation d'une queue de priorité. La priorité d'une sous-problème serait déterminée par la distance qu'il présente actuellement. Cela permettrait de résoudre en priorité les sous-problèmes les plus prometteurs, ce qui pourrait améliorer les performances du programme.